

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «АПО ЭВМиС»
Тема: **«Изучение аппаратно-программной архитектуры процессоров
семейства Intel – разработка ассемблерной вставки»**

Выполнил:
Студент 4-го курса
Группы АС-66
Горобец А.В.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками

Задание: 1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).

2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.

3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов.

Вариант задания: 5. Даны 2 строки. Совместить первую половину строки 1 с первой половиной строки 2.

Текст программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
#include <chrono>

// 1. Функция обработки строк на C++
void process_cpp(const char* str1, const char* str2, char* result) {
    int len1 = strlen(str1) / 2; // Берем половину длины первой строки
    int len2 = strlen(str2) / 2; // Берем половину длины второй строки

    strncpy(result, str1, len1); // Копируем 1-ю половину str1 в result
    strncpy(result + len1, str2, len2); // Дописываем 1-ю половину str2
    result[len1 + len2] = '\0'; // Завершаем строку нулем
}

// 2. Функция обработки строк на Inline Assembler (x86 MSVC)
void process_asm(const char* str1, const char* str2, char* result) {
    __asm {
        // --- Шаг 1: Определение длины и копирование 1-й половины str1 ---
        mov edi, str1 // EDI = адрес начала str1
        mov ecx, -1 // Устанавливаем ECX в 0xFFFFFFFF
        xor al, al // AL = 0 (ищем завершающий '\0')
        repne scasb // Сканируем str1, пока не найдем '\0'
        not ecx // Вычисляем длину: ECX = ~ECX
        dec ecx // ECX = длина str1
        shr ecx, 1 // ECX = ECX / 2 (половина длины str1)

        mov esi, str1 // ESI = источник (str1)
        mov edi, result // EDI = приемник (result)
        rep movsb // Копируем ECX байт из ESI в EDI

        // --- Шаг 2: Определение длины и копирование 1-й половины str2 ---
        push edi // Сохраняем текущий указатель на конец записанного в result
        mov edi, str2 // EDI = адрес начала str2
        mov ecx, -1 // ECX = 0xFFFFFFFF
        xor al, al // AL = 0
        repne scasb // Сканируем str2
        not ecx // ECX = длина str2 + 1
        dec ecx // ECX = длина str2
        shr ecx, 1 // ECX = ECX / 2 (половина длины str2)
        pop edi // Восстанавливаем сохраненный указатель в result
    }
```

```

        mov esi, str2    // ESI = источник (str2)
        rep movsb        // Копируем ECX байт из str2 в result

        // --- Шаг 3: Завершение строки нулем ---
        mov byte ptr[edi], 0
    }
}

int main() {
    // Настройка локализации для корректного вывода
    setlocale(LC_ALL, "Russian");

    // Входные строки (Вариант 5)
    const char* str1 = "Lab was made by AI, not me";
    const char* str2 = "by me not AI";

    char res_cpp[256] = { 0 };
    char res_asm[256] = { 0 };

    // --- 1. Проверка корректности ---
    process_cpp(str1, str2, res_cpp);
    process_asm(str1, str2, res_asm);

    std::cout << "===== " << std::endl;
    std::cout << " ИСХОДНЫЕ СТРОКИ: " << std::endl;
    std::cout << " Строка 1: " << str1 << std::endl;
    std::cout << " Строка 2: " << str2 << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << " РЕЗУЛЬТАТЫ ОБРАБОТКИ: " << std::endl;
    std::cout << " C++ результат: " << res_cpp << std::endl;
    std::cout << " ASM результат: " << res_asm << std::endl;
    std::cout << "===== " << std::endl;

    const int ITERATIONS = 10000000;

    // Замер C++
    auto start_cpp = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {
        process_cpp(str1, str2, res_cpp);
    }
    auto end_cpp = std::chrono::high_resolution_clock::now();

    // Замер ASM
    auto start_asm = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {
        process_asm(str1, str2, res_asm);
    }
    auto end_asm = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> duration_cpp = end_cpp - start_cpp;
    std::chrono::duration<double, std::milli> duration_asm = end_asm - start_asm;

    std::cout << " СРАВНЕНИЕ БЫСТРОДЕЙСТВИЯ (" << ITERATIONS << " итераций): " << std::endl;
    std::cout << " Время реализации на C++: " << duration_cpp.count() << " мс " << std::endl;
    std::cout << " Время реализации на ASM: " << duration_asm.count() << " мс " << std::endl;
    std::cout << "===== " << std::endl;

    return 0;
}

```

Вывод: в лабораторной работе проведено исследование принципа

построения, обучения и функционирования НС, реализующий однослойный персептрон Розенблатта, для решения двоичных логических функций.